Nombre: Jorge Eduardo Espal Mulgar
Carnet: 9941-24-10596

Parte I

Dado el siguiente codigo

```
#include <iostream>
using namespace std;

int main (){
int x = 8
int* p = &x;
int* q = p;

*q = 20

cout << x << endl;
cout << *p << endl;
cout << q << endl;
return 0;
}
```

Responda:

1 ¿Cual es la salida del programa?

x inicia en 8
p apunta a x
q apunta a lo mismo que p (tambien a x)
*q = 20; cambia el valor de x a 20

Su salida es:

20
20
20

2 ¿Cuantos espacios de memoria distintos existen en stack?

En stack hay 3 espacios distintos

x -> int
p -> int*
q -> int*

La "direccion a la que apuntan" p y q no es un espacio es el mismo espacio de x

3 ¿Existe aliasing?

Si hay aliasing, porque dos punteros distintos (p y q) apuntan a la misma direccion (la de x) en otras palabras si modificamos el valor de p y q modificamos el valor de x

4 Esquema de memoria

x = 20	← una direccion, por esemplo 0x100
p = &x	← p guarda la direccion de x
q = p	← q guarda la misma direccion

Parte II

```
#include <iostream>
using namespace std;
int main () {
int *p = new int;
*p = 50;

cout << "Direccion almacenada en el puntero" << p << endl;
cout << "valor almacenado en esa direccion" << *p << endl;
cout << "Direccion del puntero (&p) ;" << &p << endl;
delete p;
p = nullptr;
return 0;
}
```

Parte III

```
int *p = new int(15);
int *q = p;
delet p;
cout << *q << endl;
```

1 ¿Que tipo de error conceptual existe?
Es un error de puntero colgante debido que q esta apuntando a la memoria que ya fue liberada delete p

2 ¿Porque puede funcionar aparentemente?
Porque al hacer delete, la memoria no necesarimante se va a borrar o se sobreescribe de inmediato y el valor puede seguir temporalmente

¿Que concepto de estandar de C++ se esta violando?
Se esta probocando un comportamiento indefinido desreferenciar (*q) un puntero que apunta a memoria liberada el cual no tiene resultado garantizado

Parte IV

```
int *p = new int(10);
p = new int (30);
```

¿Existe memory leak? ¿Porque Ocurre?

Si existe, esto ocurre porque:

- p primero apunta a un int en el heap con valor 10 p = new int(30) sobre escribe la direccion anterior y no se realizo un delete asi que se queda en memoria observada

Codigo:

```
int *p = new int(10);
delete p;
p = new int (30);
delete p;
p = nullptr;
```